

# Arrays, Multidimensional Matrices and Vectors in C++

## [DEF] Definition to Remember

Arrays in C++ store contiguous collections of elements. Modern C++ provides `std::vector` for dynamic contiguous arrays with automated memory management and  $O(1)$  random access.

## 1. Concept Overview & Fundamentals

Arrays, Multidimensional Matrices and Vectors in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

\* Ze

## 2. Technical Architecture & Memory Mechanics

### C++ Runtime Execution Architecture



### Modern C++ Feature Comparison

| Modern C++ Paradigm                       | Legacy Equivalent                           | Engineering Advantage                  | Performance Impact                                                |
|-------------------------------------------|---------------------------------------------|----------------------------------------|-------------------------------------------------------------------|
| <code>**std::cin` / `std::cout`**</code>  | <code>`scanf` / `printf`</code>             | Type-safe, extensible for custom class | Buffered I/O (optimizable via <code>`ios::sy</code>               |
| <code>**References (`Type&amp;`)**</code> | Pointers ( <code>`Type*`</code> )           | Non-nullable, clean syntax without der | Direct alias; zero runtime overhead                               |
| <code>**std::vector&lt;T&gt;`**</code>    | Raw Dynamic Array ( <code>`malloc`</code> ) | Automatic resizing, bounds safety, mem | Amortized <code>**O(1)**</code> <code>push_back</code> , contiguo |
| <code>**`nullptr`**</code>                | <code>`NULL` / `0`</code>                   | Strongly typed pointer literal prevent | Zero overhead                                                     |

## 3. Core Syntax & Implementation Patterns

### Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Arrays" << "\n";

    return 0;
}
```

## 4. Real-World Applications & System Engineering

- \* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

\* Ga

## 5. Comprehensive Solved Problems & Code Walkthroughs

### Problem 1: array max min

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

#include
#include
#include
```

```
// Use
using

//
int ma
in
//
co
//
ci

//
if

//
ve
//
co
//
fo

}

in
in

//
fo

}

//
co
//
co

//
re
}
```

---

## Problem 2: array occurrences

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 */
```

```
*
* 2.
*
*
* 3.
*
*
* 4
* *
*
*
* ===
*/

#include
#include

// Use
using

//
int ma
in

//
co
//
ci

//
ve
//
co
//
fo

//
co
//
ci

//
fo

}

//
co

//
re
}
```

---

### Problem 3: array palindrome

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
* Que
```

```
* Cre
*/

/*
* ==
* CON
* ==
* 1
*
*
*
* 2.
*
*
* 3.
*
*
* 4.
* *
*
*
*
* ==
*/

#include
#include

// Use
using

//
int ma
in
//
co
//
ci

//
ve
//
co
//
fo

bo
in
//
wh

}

//
if
//
el
```

```
//  
re  
}
```

## Problem 4: array sorted check

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
//  
co  
//  
ci  
  
//  
ve  
//  
co  
//  
fo  
  
bo  
//  
fo
```

```
}  
  
//  
if  
//  
el  
  
//  
re  
  
}
```

## Problem 5: array sum average

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
  
int ma  
in  
//  
co  
//  
ci  
  
//
```

```
ve
do

//
co
//
fo

}

do

//
co
//
co

//
re

}
```

### Problem 6: copy char array pointer

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ===
 */

#include

// Use
using

void c
//
```

```
wh
}
*c
}

//
int ma
ch
ch

co

//
co
//
co

//
re
}
```

### Problem 7: delete array element

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include
#include

// Use
using

//
```

```
int main()
{
    // create a vector
    vector<int> v;

    // add elements to the vector
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);

    // print the vector
    for (int i = 0; i < v.size(); i++)
    {
        cout << v[i] << " ";
    }
    cout << endl;

    return 0;
}
```

### Problem 8: diagonal sum

### Problem Description & Objective:

### C++ Code Implementation:

## Implement

```

/*
 * Que
 * Cre
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 *
 * 2.
 *
 *

```

```
* 3.
*
*
* 4.
* *
*
*
* ===
* /

#include
#include

// Use
using

//
int ma
in
//
co
//
ci

//
ve

//
co
//
fo

}

in
in

//
fo

}

//
co
//
co

//
re
}
```

---

### Problem 9: merge sorted arrays

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include
#include

// Use
using

//
int ma
in

//
co
//
ci
//
ve
//
co
//
fo

//
co
//
ci
//
ve
//
co
//
fo

//
ve
in
```

```
//  
wh  
  
}  
  
//  
wh  
//  
wh  
  
//  
cc  
//  
fo  
//  
cc  
  
//  
re  
}
```

### Problem 10: reverse array

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 * 4.  
 * *  
 *  
 *  
 *  
 * ===  
 */  
  
#inclu  
#inclu  
#inclu
```

```
// Use
using

//
int ma
in
//
co
//
ci

//
ve
//
co
//
fo

in
//
wh

}

//
co
//
fo
//
co

//
re
}
```

---

### Problem 11: search 2d array

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
```

```
* 4.  
* *  
*  
*  
*  
*  
* ==  
*/  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
  
//  
cc  
//  
ci  
  
//  
ve  
  
//  
cc  
//  
fo  
  
}  
  
//  
cc  
//  
ci  
  
bo  
//  
fo  
  
}  
  
//  
if
```

```
//  
re  
}
```

## Problem 12: sum average 2d array

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Cre  
 */  
  
/*  
 * ===  
 * COM  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 * 3.  
 *  
 * 4.  
 * *  
 *  
 *  
 * ===  
 */  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
int ma  
in  
  
//  
co  
//  
ci  
  
//  
ve  
do  
  
//  
co  
//  
fo
```

```
}  
  
in  
do  
  
//  
co  
//  
co  
  
//  
re  
  
}
```

## 6. Exam Scoring Points & Memory Revision

### [MUST-WRITE] Must-Write Points

1. `std::vector<T>` automatically manages heap buffer reallocation as elements are appended.
2. `std::vector::push_back()` has amortized  $O(1)$  time complexity.
3. Two-pointer technique optimizes array reversal, palindrome verification, and sorted array merging.
4. Matrix operations: 2D vector representation (`vector<vector<int>>`) simplifies dynamic grid processing.
5. Spatial locality: Contiguous layout maximizes CPU cache hit ratios for sequential traversals.

### [RECALL] Quick Recall

Contiguous Buffer ->  $O(1)$  Random Access -> `std::vector` Dynamic Growth -> Two-Pointer -> Cache Friendly